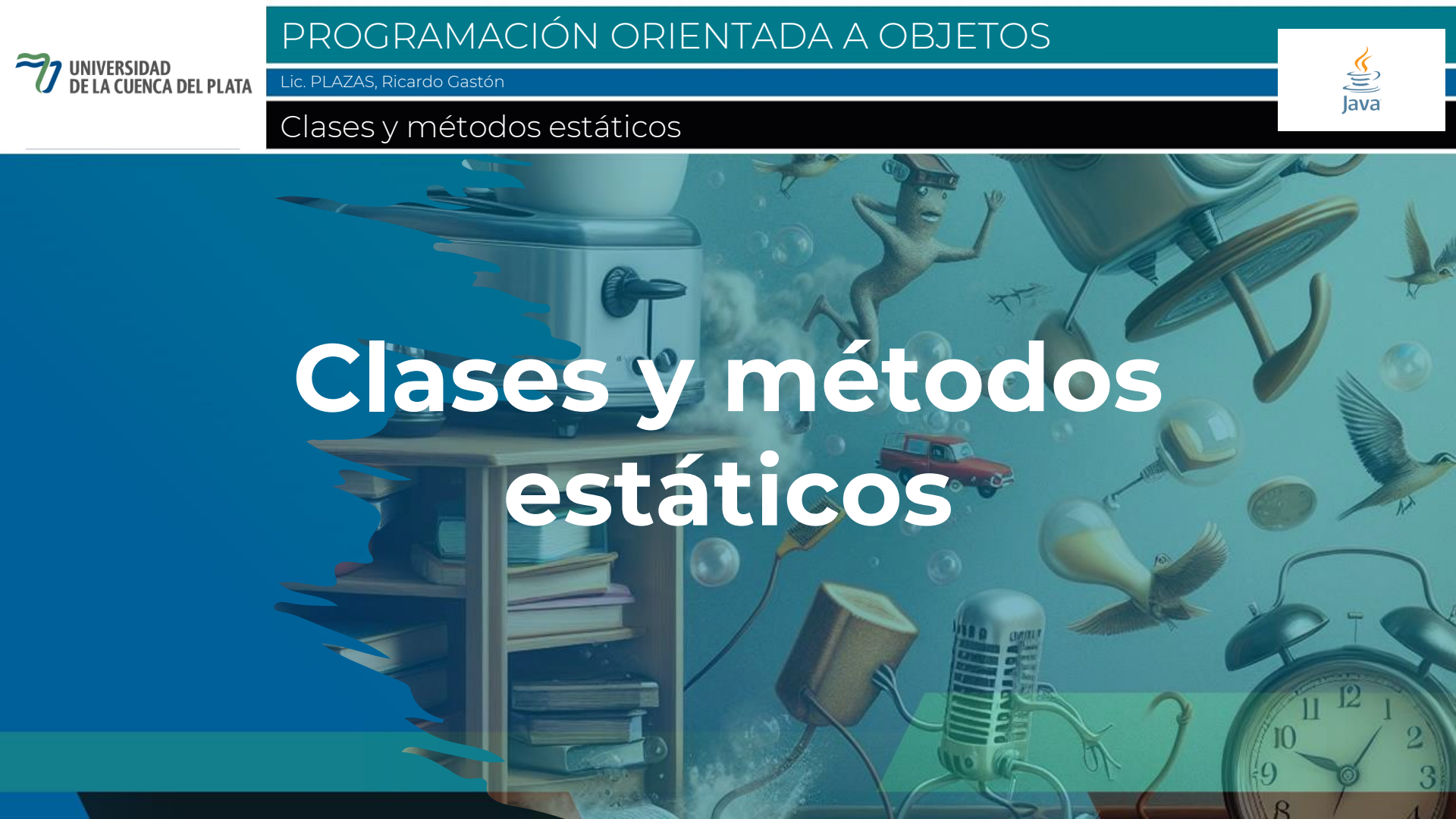




Clases y métodos estáticos

Contenido

Introducción a la POO

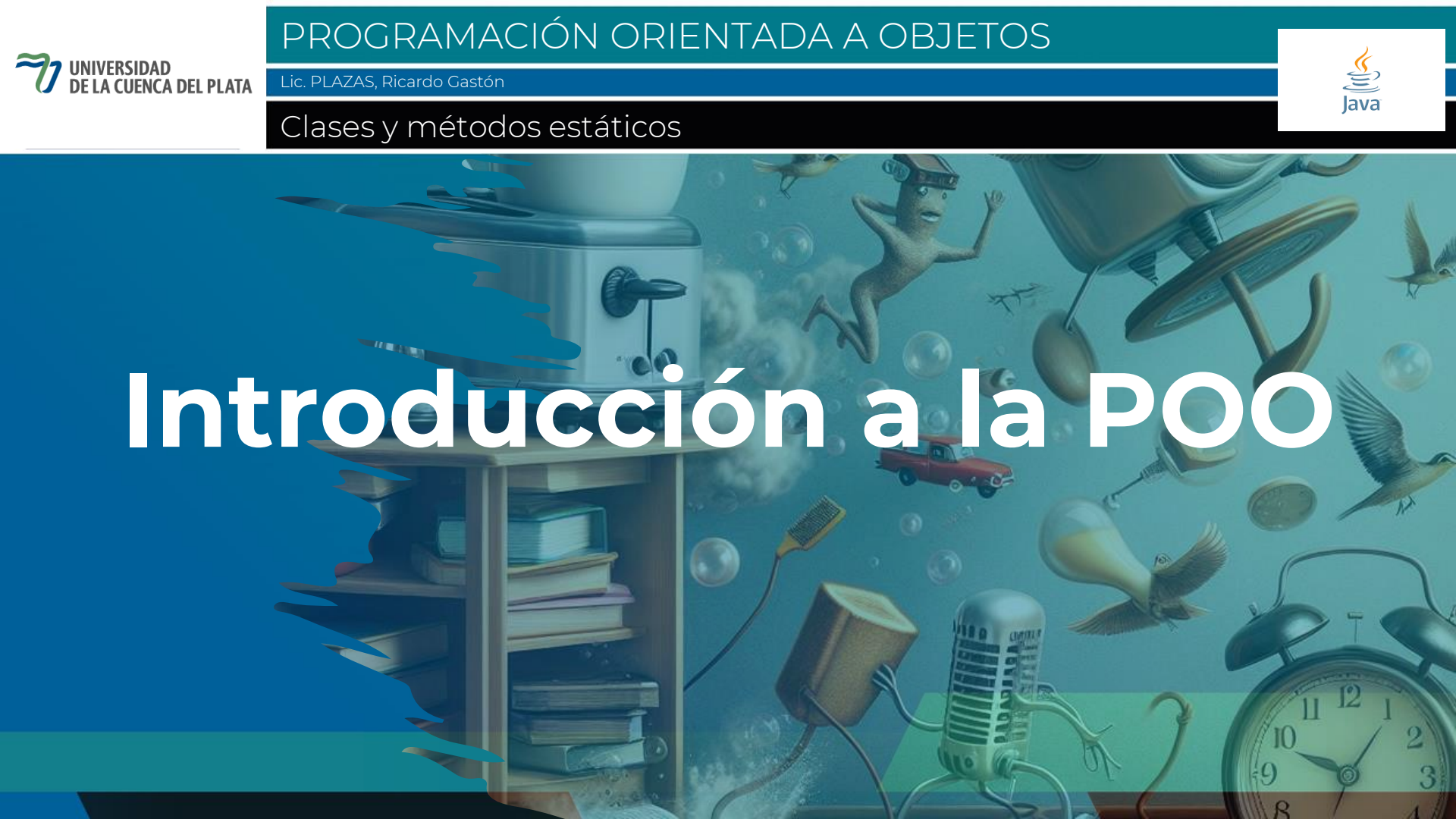
- ❖ ¿Qué es un paradigma de programación?
- ❖ ¿Qué es la programación orientada a objetos?
- ❖ Pilares de la POO.
- ❖ Principio DRY.

Clases

- ❖ ¿Qué es una clase?
- ❖ Identificadores de las clases.
- ❖ Composición de una clase.
- ❖ Sintaxis de una clase en Java.



Introducción a la POO



¿Qué es un paradigma de programación?

Un **paradigma** es una teoría o conjunto de teorías cuyo núcleo central se acepta sin cuestionar y que suministra la base y modelo para resolver problemas y avanzar en el conocimiento.

Un **paradigma de programación** define la forma, metodología o estilo con el que se resolverá un problema utilizando un lenguaje de programación.



Clases y métodos estáticos

¿Qué es la Programación Orientada a Objetos?

Es un **paradigma de programación** que propone resolver problemas a través de identificar objetos de la vida real, sus **atributos** (datos), su **comportamiento** (acciones) y las **relaciones** de colaboración entre ellos.

CuentaBancaria



Pilares de la POO

- Abstracción
- Encapsulamiento
- Herencia
- Polimorfismo



Abstracción

Permite representar conceptos del mundo real en forma de clases y objetos en el software.

Identificar Entidades Relevantes

Definir Propiedades y Comportamientos

Ocultar Detalles Internos

La abstracción nos permite simplificar la complejidad de un sistema al enfocarnos en los aspectos esenciales.

Facilita la reutilización de código al encapsular funcionalidades en clases y objetos que pueden ser utilizados en múltiples contextos.





Encapsulamiento

Es un principio de la programación orientada a objetos que consiste en ocultar los detalles internos de un objeto y exponer solo la interfaz pública para interactuar con él. Esto se logra mediante el uso de clases y la restricción del acceso a los datos internos de un objeto.



Características Encapsulamiento

Datos y Métodos: En la encapsulación, los datos (atributos) y los métodos (comportamientos) asociados a un objeto se agrupan en una clase. Los datos se mantienen privados y solo son accesibles a través de métodos públicos de la clase.

Control de Acceso: El encapsulamiento implica establecer niveles de acceso a los miembros de una clase. Los atributos privados solo son accesibles dentro de la misma clase, mientras que los métodos públicos permiten la interacción controlada con los datos internos.

Modularidad: El encapsulamiento promueve la modularidad del código al dividir el sistema en unidades independientes (clases) que se comunican a través de interfaces definidas. Esto facilita la comprensión y el mantenimiento del software.



Herencia

Permite la creación de nuevas clases (subclases) basadas en clases existentes (superclases). La subclase hereda atributos y métodos de la superclase, lo que promueve la reutilización de código y la creación de jerarquías de clases.



Herencia Características

Clase Base y Clase Derivada: Una clase base actúa como el punto de partida para la creación de una nueva clase

La subclase hereda características de la superclase y puede agregar nuevas funcionalidades o modificar el comportamiento existente.

Extensión y Especialización: Permite extender el comportamiento de una clase base mediante la adición de nuevos métodos o atributos en la subclase. También facilita la especialización, donde una subclase puede sobrescribir métodos heredados para adaptarlos a sus necesidades específicas.

Jerarquía de Clases: Las clases pueden organizarse en una jerarquía, donde las clases más generales están en niveles superiores y las clases más especializadas están en niveles inferiores. Esto permite una representación más precisa y estructurada del dominio del problema.



Polimorfismo

Permite que un objeto pueda presentarse de diversas formas. En la programación orientada a objetos, el polimorfismo se refiere a la capacidad de un objeto de comportarse de diferentes maneras dependiendo del contexto en el que se encuentre.



Polimorfismo Beneficio

Flexibilidad: Permite escribir código más genérico y flexible, ya que los objetos pueden adaptarse dinámicamente a diferentes situaciones.

Facilita la Extensión: Facilita la extensión del software, ya que nuevas clases pueden ser añadidas al sistema sin afectar el código existente.



Principio DRY

*"Toda **pieza de conocimiento** debe tener una representación **única**, **inequívoca** y **fidedigna** dentro de un sistema."*

The pragmatic programmer - Dave Thomas



PIEZA DE CONOCIMIENTO

Funcionalidad
precisa dentro del
contexto de
negocio o un
algoritmo
concreto.



UNICA

No debe existir
otra repetición



INEQUIVOCA

Sólo puede ser
interpretada,
entendida o
explicada de una
manera



FIDEDIGNA

Confiamos en que
es correcta

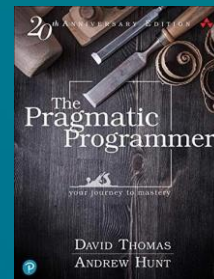
Principio DRY

“Many people took it [the DRY principle] to refer to code only: they thought that DRY means ‘don’t copy-and-paste lines of sources.’ [...] DRY is about the duplication of knowledge, of intent. It’s about expressing the same thing in two different places, possibly in two totally different ways”.

The pragmatic programmer - Dave Thomas

La idea del principio DRY es simple:

Cuando ocurra un cambio no deberíamos necesitar actualizar múltiples cosas en paralelo



Clases

¿Qué es una CLASE?

Una clase es una **descripción** de un **conjunto de objetos** que comparten los mismos **atributos, métodos, relaciones** y **semántica** en un determinado **contexto**.

Una clase es una implementación de una abstracción.

Identificadores de clases

A la hora de nombrar clases debemos seguir las siguientes convenciones:

Grafía pascal (UpperCamelCase)

La primera letra del identificador y la primera letra de las siguientes palabras concatenadas están en mayúsculas.

Uso de sustantivos

Los nombres de clases se escriben con sustantivos, ya que representan a objetos.

Nombres descriptivos

Como todo identificador, debe describir con la mayor exactitud posible a lo que representa.

Ejemplos

HistoriaClinica, Mascota, JugadorDeFutbol, LibroDiario, Transferencia, DocenteAdjunto, ExamenFinal.

Composición de una clase

Atributos

Representan **características** que son compartidas por todos los objetos de una clase.

Definen el rango de valores que puede tomar cada una de las propiedades de un objeto.

Utilizar notación **camelCase** y **sustantivos**.



Métodos

Un método es **la implementación de una operación**.

Una operación es **una abstracción de algo que puede hacer un objeto** y que es compartido por todos los objetos de esa clase.

Utilizar notación **camelCase** y **verbos**.



Composición de una clase

Relaciones

Las clases se conectan entre sí a través de relaciones y **colaboran** para realizar un comportamiento mayor.

Los objetos de clases relacionadas pueden interactuar entre sí.

Semántica

La semántica es el **significado** que se le da a la clase y a sus relaciones dentro del contexto de negocio.



Sintaxis de una clase en Java

```
public class Persona {  
  
    // ATRIBUTOS  
    public String apellido;  
    public String nombre;  
    public int edad;  
    public static String datoEstatico;  
  
    // CONSTRUCTOR  
    public Persona(){  
  
        this.apellido = "Doe";  
        this.nombre = "John";  
        this.edad = 20;  
    }  
  
    // MÉTODOS  
    public void saludar(){  
  
        System.out.println("Hola, soy " + this.nombre +  
                             " " + this.apellido);  
        System.out.println("Tengo " + this.edad + " años.");  
    }  
  
    public static String metodoEstatico(){  
  
        return Persona.datoEstatico;  
    }  
}
```

Modificadores de clase

NOMBRE	DESCRIPCIÓN
(*) public	Accesible desde cualquier paquete.
(*) default (sin modificador)	Accesible sólo desde el mismo paquete.
final	La clase no puede ser derivada.
abstract	La clase no puede ser instanciada.

Sintaxis atributos en Java

Modificadores de atributos

```
// ATRIBUTOS
```

```
public String apellido;  
public String nombre;  
public int edad;  
public static String datoEstatico;
```

NOMBRE	DESCRIPCIÓN
(*) public	Accesible desde cualquier paquete.
(*) protected	Accesibles desde el mismo paquete y derivadas.
(*) default (sin modificador)	Accesible sólo desde el mismo paquete.
(*) private	Accesible sólo desde la misma clase.
final	Una vez asignada, no podrá modificarse.
static	Pertenece a la clase y no a las instancias

Sintaxis métodos en Java

Modificadores de métodos

```
// MÉTODOS
public void saludar(){

    System.out.println("Hola, soy " + this.nombre +
                       " " + this.apellido);
    System.out.println("Tengo " + this.edad + " años.");
}

public static String metodoEstatico(){

    return Persona.datoEstatico;
}
```

NOMBRE	DESCRIPCIÓN
(*) public	Accesible desde cualquier paquete.
(*) protected	Accesibles desde el mismo paquete y derivadas.
(*) default (sin modificador)	Accesible sólo desde el mismo paquete.
(*) private	Accesible sólo desde la misma clase.
final	No puede ser sobrescrito por clases derivadas.
static	Pertenece a la clase y no a las instancias.
abstract	No tiene implementación y debe ser sobrescrito.

Ejercitación